

纹理压缩

1 目的

- (1) 减少显存占用;
- (2) 降低数据传输的带宽。

2 纹理压缩格式

2.1 S3TC/DXTC

2.1.1 介绍

S3TC 是 S3 公司提出 Texture Compression 算法,也叫做 DXTC(Direct Texture Compression)。基本思想为块压缩,以原始大小存储一些颜色,然后以不同的编码方案存储其他颜色。如图1所示,将未压缩的纹理分解成 4×4 的纹理块,对每个块进行压缩,此处要求被压缩的纹理的维度必须是 4 的倍数。

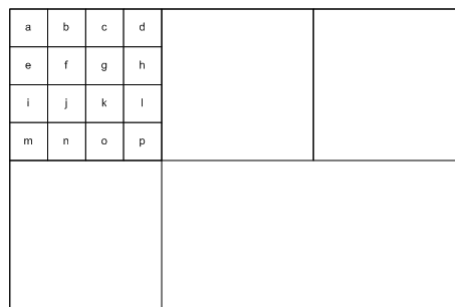


图 1: Block Compress

2.1.2 BC1

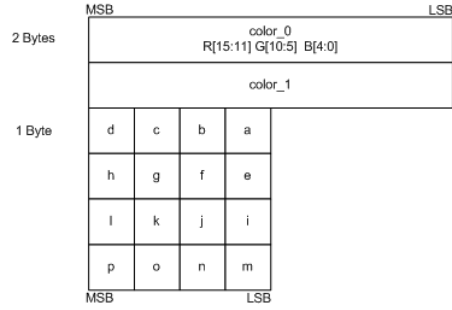


图 2: BC1

如图2所示，BC1 压缩格式对 4×4 的纹理块进行处理，BC1 并不是存储 16 种颜色，而是存储两种颜色（*color_1* 和 *color_2*）以及 16 个 2 位颜色指数，两种颜色都是用 5:6:5（5 位红色、6 位绿色、5 位蓝色）进行编码，将所需内存从 48 字节减少到 8 字节。能表示如下四种颜色：

color_0

color_1

$$color_2 = 2/3 * color_0 + 1/3 * color_2$$

$$color_3 = 1/3 * color_0 + 2/3 * color_2$$

四种分别对应 00、01、10 和 11，纹理块中的每种颜色和四种颜色比较后，存储最接近的颜色在纹理块中。

该算法同样适用于包含 1 位 alpha 的数据，进行如下设置：

color_0

color_1

$$color_2 = 1/2 * color_0 + 1/2 * color_2$$

$$color_3 = 0$$

2.1.3 BC2

相比于 BC1，BC2 更加关注低一致性的 alpha 数据。BC2 将 alpha 存储为单独的 4 位值，将 64 字节压缩到 16 字节，如图3所示。



图 3: BC2

2.1.4 BC3

BC3 用于存储高一致性的颜色数据（对于一致性较低的 alpha 数据使用 BC2）。BC3 的颜色数据格式与 BC1 相同，对于 alpha 数据，BC3 存储两个参考 alpha (α_0 和 α_1)，以及 16 个 3 位颜色指数，将 64 字节压缩到 16 字节，如图4所示。

BC3 的 alpha 查找表共有 8 个元素，除了 α_0 和 α_1 ，计算方法如下：

```

1 if( alpha_0 > alpha_1 )
2 {
3     // 6 interpolated alpha values.
4     alpha_2 = 6/7*alpha_0 + 1/7*alpha_1; // bit code 010
5     alpha_3 = 5/7*alpha_0 + 2/7*alpha_1; // bit code 011
6     alpha_4 = 4/7*alpha_0 + 3/7*alpha_1; // bit code 100

```

```

7   alpha_5 = 3/7*alpha_0 + 4/7*alpha_1; // bit code 101
8   alpha_6 = 2/7*alpha_0 + 5/7*alpha_1; // bit code 110
9   alpha_7 = 1/7*alpha_0 + 6/7*alpha_1; // bit code 111
10  }
11  else
12  {
13      // 4 interpolated alpha values.
14      alpha_2 = 4/5*alpha_0 + 1/5*alpha_1; // bit code 010
15      alpha_3 = 3/5*alpha_0 + 2/5*alpha_1; // bit code 011
16      alpha_4 = 2/5*alpha_0 + 3/5*alpha_1; // bit code 100
17      alpha_5 = 1/5*alpha_0 + 4/5*alpha_1; // bit code 101
18      alpha_6 = 0;                          // bit code 110
19      alpha_7 = 255;                         // bit code 111
20  }

```



图 4: BC3

2.1.5 BC4

BC4 用于存储单通道数据，如灰度图。BC4 存储两种参考颜色 (`red_0` 和 `red_1`)，以及 16 个 3 位颜色指数，将 16 字节压缩到 8 字节，如图5所

示。

BC4 和 BC3 一样，通过检查两个参考颜色来确定内插颜色值的数量。如果 red_0 大于 red_1 ，BC4 内插 6 个颜色值；否则，内插 4 个颜色值。当 BC4 内插 4 个颜色值，会设置两个额外的颜色值（0.0f 表示完全透明，1.0f 表示完全不透明）。BC4_UNORM 存储 $[0, 1]$ 范围内的浮点数据，BC4_SNORM 存储 $[-1, 1]$ 范围内的浮点数据。

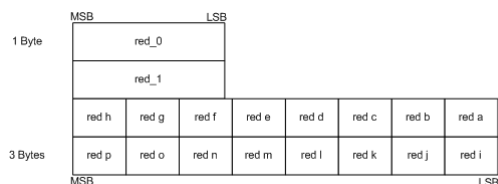


图 5: BC4

2.1.6 BC5

BC5 用于存储两个颜色分量的数据，每个分量存储 2 种参考颜色 (red_0 、 red_1 、 $green_0$ 、 $green_1$) 和 16 个三位颜色指数，将 32 个字节压缩到 16 个字节，如图6所示。每个分量的插值策略与 BC4 相同，BC5_UNORM 存储 $[0, 1]$ 范围内的浮点数据，BC5_SNORM 存储 $[-1, 1]$ 范围内的浮点数据。

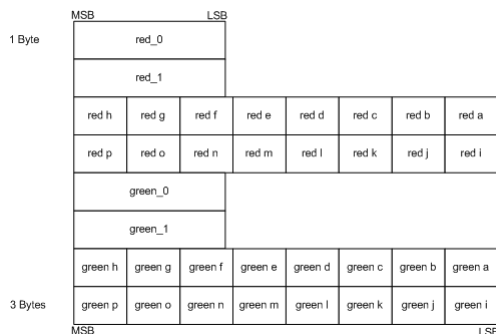


图 6: BC5

2.1.7 BC6H

BC6H 用于存储 HDR 颜色数据，存储三个颜色通道，每个通道存储 16 位值，不支持 alpha 通道。16 位浮点格式分为以下两种形式：

- (1) UF16: 无符号浮点数，包含 5 个指数位和 11 个尾数位；
- (2) SF16: 有符号浮点数，包含 1 个符号位、5 个指数位和 10 个尾数位。

BC6H 处理的对象仍然是 4×4 的纹理块，BC6H 将其压缩成 128bit 的数据，处理后的 BC6H 块包含 mode bits、compressed endpoints、compressed indices 和可选的 partition index。

(1) BC6H 对 4×4 的纹理块的处理模式分为单分区和双分区，单分区只有两个 color，用于插值；双分区各有两个 color；

(2) BC6H 总共有 14 种模式，如图7所示；

(2.1) 模式 1-10 用于双分区，mode bit 可以为 2 或 5，以模式 3 为例，表示 color endpoint 的精度 level 为 11，存储变换后的 endpoint 的 delta value 的精度分别为 5bit (red)、4bit (green)、4bit (blue)；模式 10 没有使用 delta 压缩，直接存储 4 个 color endpoint；

(2.2) 模式 11-14 用于单分区，mode bit 固定为 5，以模式 11 为例，没有使用 delta 压缩，直接存储两个 color endpoint；

Mode	Partition Indices	Partition	Color Endpoints	Mode Bits
1	46 bits	5 bits	75 bits (10.555, 10.555, 10.555)	2 bits (00)
2	46 bits	5 bits	75 bits (7666, 7666, 7666)	2 bits (01)
3	46 bits	5 bits	72 bits (11.555, 11.444, 11.444)	5 bits (00010)
4	46 bits	5 bits	72 bits (11.444, 11.555, 11.444)	5 bits (00110)
5	46 bits	5 bits	72 bits (11.444, 11.444, 11.555)	5 bits (01010)
6	46 bits	5 bits	72 bits (9555, 9555, 9555)	5 bits (01110)
7	46 bits	5 bits	72 bits (8666, 8555, 8555)	5 bits (10010)
8	46 bits	5 bits	72 bits (8555, 8666, 8555)	5 bits (10110)
9	46 bits	5 bits	72 bits (8555, 8555, 8666)	5 bits (11010)
10	46 bits	5 bits	72 bits (6666, 6666, 6666)	5 bits (11110)
11	63 bits	0 bits	60 bits (10.10, 10.10, 10.10)	5 bits (00011)
12	63 bits	0 bits	60 bits (11.9, 11.9, 11.9)	5 bits (00111)
13	63 bits	0 bits	60 bits (12.8, 12.8, 12.8)	5 bits (01011)
14	63 bits	0 bits	60 bits (16.4, 16.4, 16.4)	5 bits (01111)

图 7: BC6H 模式

(3) 对于双分区，BC6H 共有 32 种分割情况，如图8所示；图7中的

Partition 对应选择哪种分割情况，图8中有 1 被添加了下划线和加粗，表示使用了端点对称性的技巧：假设用 3bit 来表示 color1 和 color2 的插值权重，如果权重最高位为 0，那么我们舍弃这个 0，像素的权重位可以节省 1 位，如果最高位为 1，可以交换 color1 和 color2，那么权重最高位会由 1 变成 0，还是可以节省 1 位：

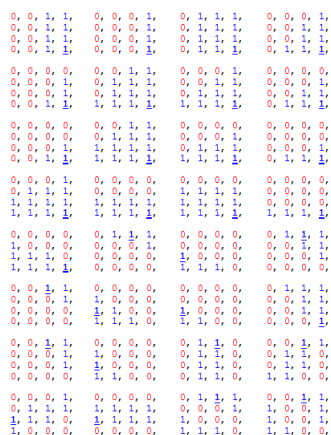


图 8: BC6H 分割情况

BC7 格式是用于 RGB 和 RGBA 数据的高质量压缩的纹理压缩格式。BC7 对 4×4 的纹理块进行处理, 用 16 字节 (128 位) 的固定块进行表示。

图 9: BC7 Block

种：

(1) 不包含 alpha 的 BC7 块, BC7 块由模式位、分区位、压缩的 endpoint、压缩索引和可选的 P 位组成，其中 endpoint 为 RGB 格式，对于源数据中的所有纹素，alpha 值均为 1.0；

(2) 包含 alpha 的 BC7 块, BC7 块由模式位、压缩的 endpoint、压缩索引、可选的 P 位和分区位组成，endpoint 为 RGBA 格式，alpha 部分和 RGB 分量一起进行插值；

(3) alpha 部分和 RGB 部分单独处理的 BC7 块, BC7 块由模式位、旋转位、压缩的 endpoint、压缩索引和可选的索引选择位组成，alpha 部分和 RGB 部分分别进行插值。

BC7 块共有 8 种模式，且一个 BC7 块可以包含多个 endpoint 对。endpoint 用"RGBP"表示，其中"P"表示 endpoint 各通道共享的最小有效位。比如，"RGB 5.5.5.1"表示 endpoint 实际精度为 RGB 6.6.6；对于具有 alpha 通道的 endpoint，"RGB 5.5.5.5.1"表示 endpoint 实际精度为 RGB 6.6.6.6。

下面，简单介绍下 BC7 块的模式 0，其余见 BC7 模式：

(1) 模式 0

- 只有颜色分量（无 alpha）
- 每个块有 3 对 endpoint
- 每个 endpoint 的存储格式为 RGBP 4.4.4.1
- 3 位索引
- 4bit 的分区

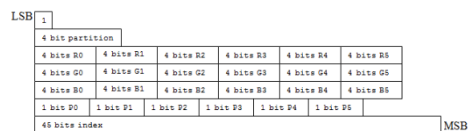


图 10: BC7 Mode0

3 Random-Access Neural Compression of Material Textures

3.1 目的

用神经网络进行纹理压缩，减少纹理的显存占用和带宽占用，提高渲染效率。

3.2 算法介绍

3.3 texture set

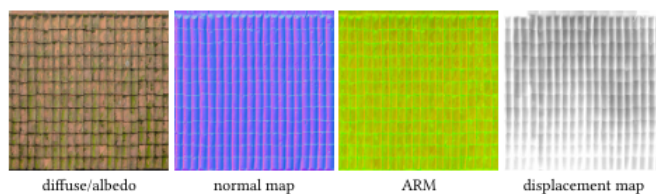


图 11: Texture Set

如图11所示，处理对象为一个 group 内的多个 texture，称作 texture set。它包含 diffuse albedo、normal map、ARM (ambient occlusion、roughness、metallic) 和 displacement map。其中假设所有纹理的 width 和 height 都相同。

3.4 算法流程

如图12所示，整个算法流程分为：

- (1) Feature Pyramid
- (2) Simulated Quantization
- (3) Sampling and Concatenation
- (4) Network
- (5) Predicted Mip Level

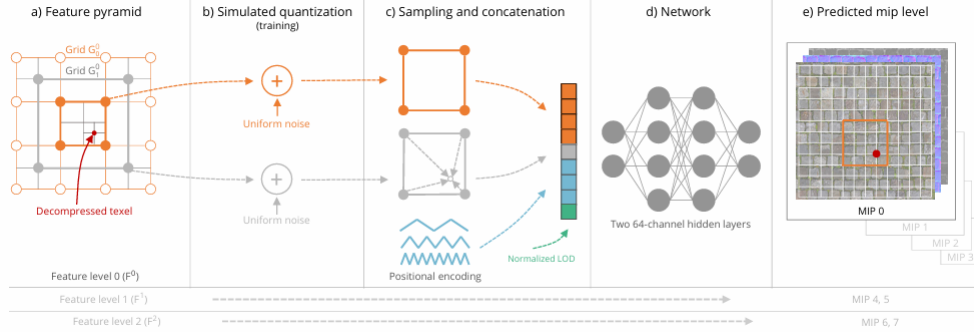


图 12: 算法流程

3.4.1 Feature Pyramid

如图12(a) 所示, 压缩格式为特征金字塔, 对于第 j 层, 其 feature level 记为 F^j , 每层包含一对 2D grid, 记作 G_0^j 和 G_1^j 。每层的 G_0 为高分辨率 grid, 用于保持高频细节; G_1 为低分辨率 grid, 提高低频内容的重建质量。

grid 的 cell 中存储量化隐向量值的特征向量, 用于预测 mip level。每层的特征向量对应多个 mip level, 相较于传统的 mipmap 方法, 存储占用从 33% 减少到 6.7%, 甚至更少。

如图13所示, 对应 1024×1024 的 texture set。grid 的分辨率远远低于纹理分辨率, 对于整个 mip chain 有较高的压缩率。

Table 1. Compressed representation of a mip chain through feature levels and low resolution grids for a 1024×1024 texture set.

Feature level F^j	G_0^j grid resolution	G_1^j grid resolution	Predicted mip levels
0	256×256	128×128	0,1,2,3
1	64×64	32×32	4,5
2	16×16	8×8	6,7
3	4×4	2×2	8,9,10

图 13: Feature Pyramid

3.4.2 Simulated Quantization

论文没有采用熵编码, 而是采用了:

- 固定的量化区间 (quantization bins): 每个 grid k 对应 Q_k

- 添加噪声模拟量化误差（对特征添加均匀噪声 $U(-\frac{Q_k}{2}, \frac{Q_k}{2})$ ）
- 通过反向传播优化量化后的特征值

训练策略：

（1）在训练时，如果只在前向传播阶段对特征值进行 clamp，那么反向传播时这些特征值可能会”漂移“出这个范围，导致训练结果不佳。

（2）本论文的处理方法是在反向传播时也对特征值进行 clamp，使得整个训练过程的特征值始终保持在量化区间内，从而提高压缩质量。

对于 grid G_k^j ，论文使用了非对称量化区间 $[-\frac{N_k-1}{2}Q_k, \frac{N_k}{2}Q_k]$ ，其中 $N_k = 2^{B_k}$ 为量化 level 数。这样设计，让 0 值恰好落在量化区间的中心，这在量化 level 小于等于 4 的时候会产生更好的结果。论文中将 Q_k 设置成 $\frac{1}{N_k}$ 。

对齐 0 值的重要性：

- 在神经网络压缩中，很多特征值接近 0，尤其是在稀疏或低频区域。
- 如果 0 值落在 bin 的边缘，量化时可能会偏移，导致误差。
- 提高压缩质量，尤其是低比特率

举例（4 个量化等级， $N_k = 4$ ）：

- $Q_k = \frac{1}{4} = 0.25$
- 区间范围为 $[-\frac{3}{2} \cdot 0.25, \frac{4}{2} \cdot 0.25] = [-0.375, 0.5]$
- 4 个 bin 依次为 $[-0.375, -0.125), [-0.125, 0.125), [0.125, 0.375), [0.375, 0.5]$
- 0 值恰好落在第二个 bin 的中心

3.4.3 Sampling and Concatenation

（1）Feature Interpolation

论文在同时考虑复杂度和重建质量的前提下，使用两种不同的策略对 grid 进行采样。对高分辨率 grid G_0 应用可学习插值，对低分辨率 grid G_1 使用双线性插值。

对于高分辨率网格的可学习插值，我们将相邻的 4 个特征向量进行拼接，然后用位置编码的相位信息进行重建高频细节。

此处使用拼接而不是加权求和的原因：

- 信息丢失：加权求和会融合特征，可能掩盖局部差异
- 位置不敏感：求和后的结果无法保留每个采样点的独立性

但是拼接也会导致网络的输入增多，增加网络负担，所以对于低分辨率网格采用双线性插值。

(2) Tiled Positional Encoding

位置编码的作用主要在于：

- 补偿低分辨率 grid 的频率限制：低分辨率网格只能表示低频信息
- 打破 MLP 的低频偏差：MLP 倾向于拟合平滑、低频信号，位置编码通过引入周期性函数（如三角函数）将空间坐标转为高频信号，让网络能学习更复杂的空间变化
- 提高网络的空间敏感性：拼接位置编码后，MLP 可以对不同位置做出不同的预测

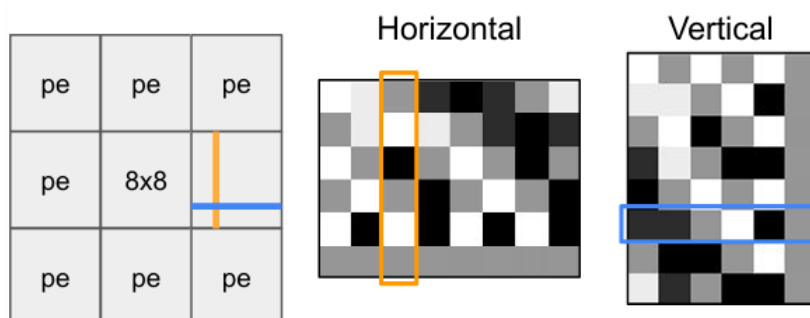


图 14: Position Encoding

如图14所示，处理的基本对象为 8×8 的 texel 块。这里取 8 是因为 texture set 的分辨率为 1024×1024 ，从最小网格上采样到最高分辨率对应的最大上采样因子为 8。

这里采用的编码方式为三角波编码（参考 NVIDIA 的 Neural Radiance Cache 论文），相比于正弦和余弦编码，三角波计算更高效，且在视觉上没有明显的质量损失。每个 texel 被编码成 6 个水平的标量值和 6 个垂直的标量值，其中水平编码和垂直编码的最后一个值均为常量。

综上，拼接后的向量包括：

- 从高分辨率网格 G_0 采样的 4 个特征向量，维度记为 C_0
- 从低分辨率网格 G_1 插值得到的特征向量，维度记为 C_1
- 12 个位置编码
- 1 个归一化的 LOD 值

最终输入维度为 $4C_0 + C_1 + 12 + 1$ 。

3.4.4 Network

整个网络是一个简单的 MLP，除了输入层和输出层，还包括两个中间层，通道数为 64。对于输出层的激活函数，作者提出经测试后 GELU 的效果最好。但是为了减少 GELU 的计算开销，作者提出了”hardGELU”作为估计，类似于 hard Swish：

$$\text{hardGELU}(x) = \begin{cases} 0, & \text{if } x < -\frac{3}{2} \\ x, & \text{if } x > \frac{3}{2} \\ \frac{x}{3}(x + \frac{3}{2}), & \text{otherwise} \end{cases}$$

3.5 实现细节

3.5.1 压缩过程

自定义 CUDA 实现：

- 使用半精度 Tensor Core 进行运算，大幅提升训练速度
- 将网络所有层都放进一个 CUDA kernel 中，避免中间数据写入共享内存或外部内存

采样策略：

- 每个 batch 处理 8 个随机采样的来自相同 LOD 的 256×256 的 texel 块
- 每个 batch 随机选择 LOD 层级，策略如下： $LOD = \lfloor -\log_4 X \rfloor, X \sim U(0, 1)$

- 为了防止低分辨率层级被忽略，5% 的 batch 使用均匀分布采样所有 mip 层级。

学习策略：

- 初始学习率为 0.01，网络权重为 0.005
- 使用余弦退火（cosine annealing）逐步降低学习率至 0

3.5.2 解压缩过程

SIMD 协同矩阵运算

- Tensor Core 使用 SIMD 协同矩阵进行运算，其中矩阵按 SIMD lane 排布
- 在光追的 shader 中无法访问共享内存，所以使用 SIMD-wide shuffle intrinsics 将数据在寄存器中进行重排

着色语言与编译器支持

- 使用 Slang 着色语言编写 shader
- 对 Direct3D 编译器进行修改，生成 NVVM 调用，用于矩阵操作和 shuffle intrinsics
- Vulkan 通过拓展 $NV_{cooperate_matrix}$ 和 $KHR_{shader_ubgroup}$ 实现

SIMD Divergence

在 GPU 上，多个线程（称作 SIMD lanes）通常会并行执行相同的指令，但是每个线程处理的数据不同，这要求：

- 所有线程使用相同的网络权重，才能高效利用 Tensor Core 进行矩阵运算
- 如果不同 lanes 需要不同的网络（例如对应不同材质），就会发生 SIMD Divergence，导致性能下降

因为不同材质对应不同的 MLP 网络，不同光线可能命中不同材质，这就会出现 SIMD Divergence。

作者提出了以下优化策略：

- (1) 在一个 SIMD warp 内，识别所有材质对应的网络
- (2) 对网络进行遍历，只在对应的 lane 上进行运算
- (3) 以此保持每次推理的权重一致性，并可以使用 Tensor Core 进行加速

SIMD Divergence 会严重影响性能，作者建议使用 SER (Shader Execution Reordering) 和 TSU (Thread Subgroup Unification) 来减少性能损失。

3.5.3 过滤

NTC 将纹理压缩成 grid，然后使用 MLP 进行预测，将预测结果作为纹理。这种方法有两个限制：

- 训练只在离散的 texel 上进行，没有在 texel 和 texel 中间的位置进行训练，无法预测中间位置的纹理。
- 无法使用 GPU 硬件的三线性过滤

作者提出了随机过滤 (Stochastic Filtering) 来替代三线性过滤，其实现方式为：

- (1) 对采样坐标 (u, v) 添加噪声，使用均匀分布模拟双线性过滤，使用正态分布模拟高斯过滤。
- (2) 对 LOD 也添加扰动，实现 mip 层级的平滑过渡

作者同时提出了随机过滤会增加渲染图像的噪声，可以通过 FSR、XeSS 和 DLSS 来进行降噪。

3.6 参考

Random-Access Neural Compression of Material Textures
RTXNTC
Real-time Neural Radiance Caching for Path Tracing

4 参考

Compressed GPU texture formats

Unity 对各平台纹理格式的建议
块压缩 (Direct3D 10)
Direct3D 11 中的纹理块压缩